

also the Store Manager app for Google Shopping Express.

Outside Google, the teams that have started using Flutter are Hamilton Musical, Alibaba, AppTree, Abbey Road, etc. You can find a list of such apps at <https://flutter.io/showcase/>.

'Hello World' in Flutter

The easiest way to get introduced to any language is to understand the 'Hello World' code. So let's get started.

```
import 'package:flutter/material.dart';

void main() => runApp(new MaterialApp(
  title: 'First Flutter App',
  home: MyApp(),
));

class MyApp extends StatelessWidget {
  // This widget is the root of your
  application.
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Demo App'),
      ),
      body: Center(
        child: Text('Hello World!'),
      ),
    );
  }
}
```

Importing the *material.dart*

package: This is a pre-defined file to give the look and feel of material design to the app. We can materialise the app by using the widgets from the package (e.g., AppBar, Scaffold, etc).

MaterialApp: This consists of 'home', 'route', etc, as its parameters.

StatelessWidget: These widgets store the values of their parent widgets and use these values when the application is built.

Scaffold: This is a widget under the material app. It acts as a layout for the major material components.

MaterialApp can be built and run on an emulator or on a real device, similar to Android or iOS applications.

Yes! We have completed our first Flutter app and it'll look exactly as shown in Figure 2.

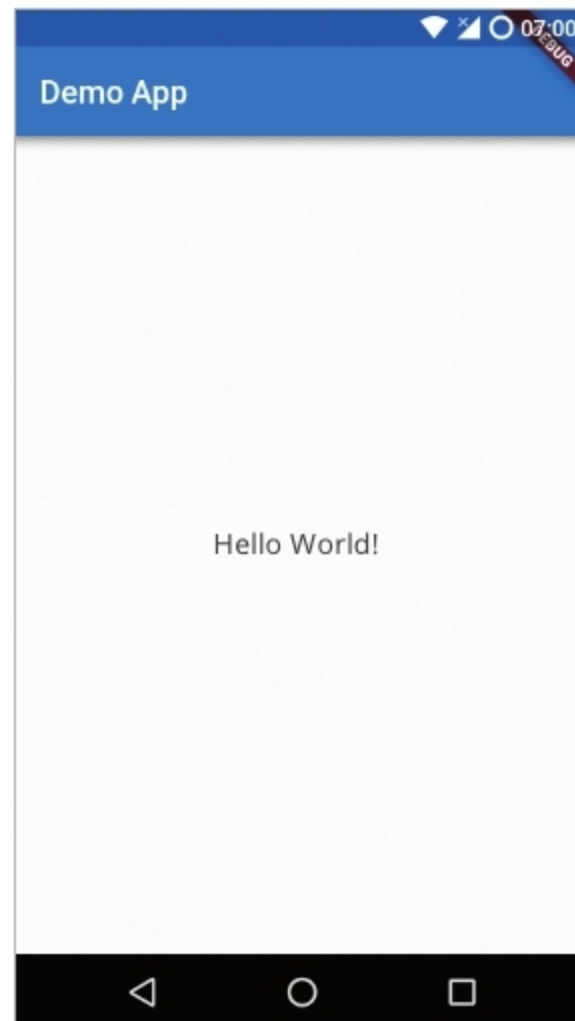


Figure 2: Hello World!

Syntax and components

As we know, Flutter uses the Dart language for development. So let's look at the basic syntax and usability of the various components of a Flutter app. Since we can't cover all of it, I recommend that you go to the Flutter website and read the documentation, which is pretty good and also try out the Flutter Codelabs. They are very interesting and will help you learn faster.

Widgets

In Flutter, everything is a widget! As a result, it gets a lot easier for developers to visualise their applications in widgets.

The most commonly used widgets are listed below.

- **Text:** It lets you create styled text in your application:

```
Text(
  'Hello World!',
  textAlign: TextAlign.center,
```

```
style: new TextStyle(fontStyle:
  FontStyle.italic),
)
```

- **Row, Column:** This let's you create horizontal and vertical flexible layouts:

```
Row(
  mainAxisAlignment: MainAxisAlignment.
  center,
  children: <Widget>[
    Icon(Icons.cake),
    Text('Happy Birthday!')
  ],
)
```

```
Column(
  mainAxisAlignment: MainAxisAlignment.
  center,
  children: <Widget>[
    Icon(Icons.cake),
    Text('Happy Birthday!')
  ],
)
```

Stack: As opposed to Row and Column, Stack allows you to place widgets on top of one another.

Container: With its properties of padding, margins and various constraints, it lets you create a rectangular visual element for your application.

You can read about the basic widgets like Image, Icon, RaisedButton, etc, at <https://flutter.io/widgets/basics/>.

Stateful and stateless widgets: Widgets are divided into two types — stateful and stateless. You can select either depending upon the need of the application. So how do they differ?

To state it simply, stateless widgets are those that do not change with the changing state. But what is state? It is basically the information to be displayed. If you want to make changes to the content displayed by a stateless widget, you need to create a brand new widget.

Stateless widgets always build the same way, given a particular configuration.